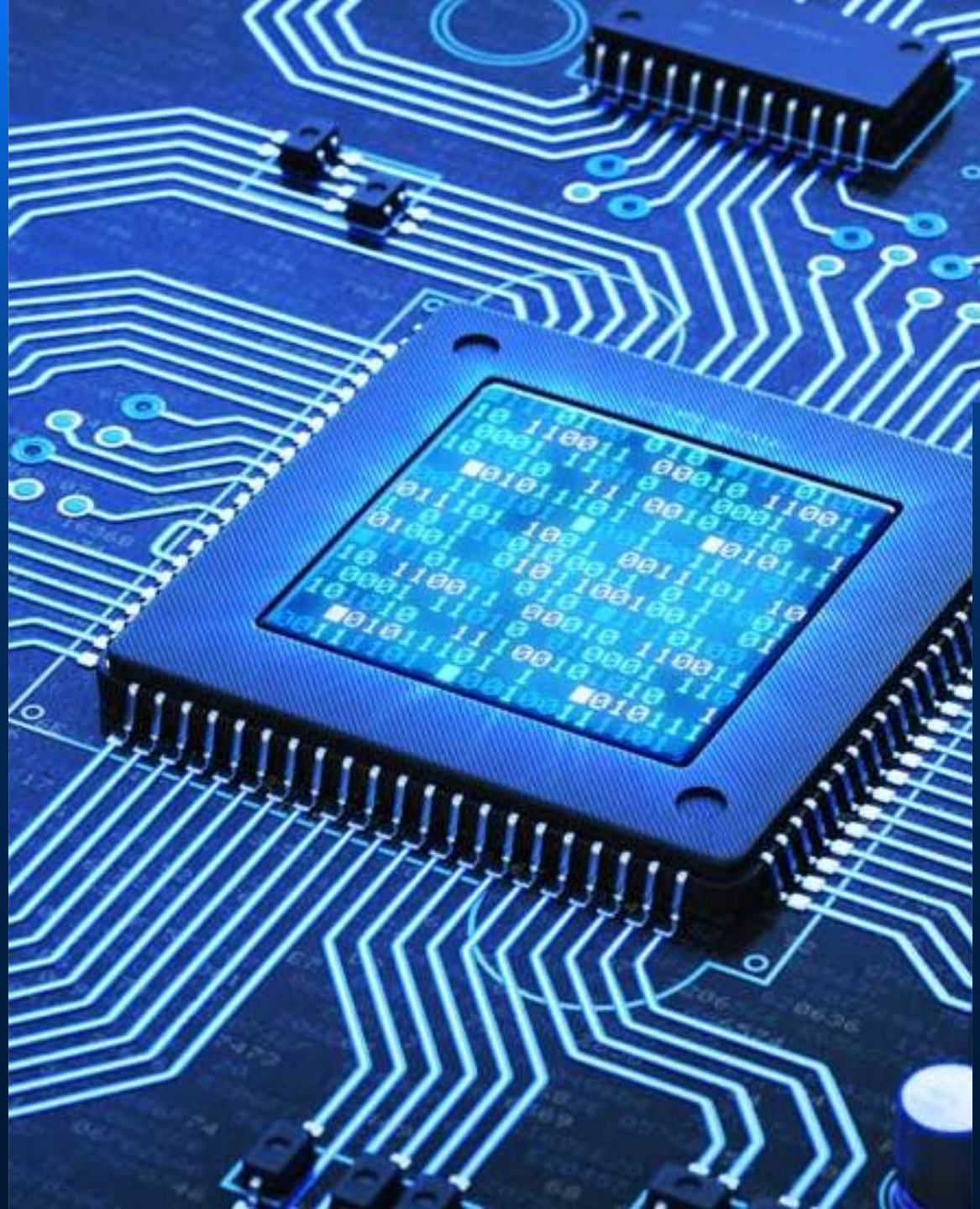


Computer Architecture

Session 8

By: Eng. Abdulkareem Mohammed

Senior HW Digital Design Engineer

This presentation is prepared from:

ARM® AMBA® 5 APB Protocol Specification.

Agenda

AMBA APB protocol

- # 1 Introduction
- # 2 Signal Descriptions
- # 3 Transfers
- # 4 Operating States

1 Introduction

1 Introduction

- APB is a **low-cost** interface, optimized for min power consumption and reduced complexity.
- APB interface is not pipelined and is a simple, synchronous protocol.
- Every transfer takes at least **two cycles** to complete.
- APB interface is designed for accessing the programmable control registers of peripheral devices.
- APB peripherals are typically connected to the main memory system using an APB bridge.
- APB transfers are initiated by an APB bridge (as a Requester).
- APB peripheral (as a Completer) responds to requests.

2 Signal Descriptions

2 Signal Descriptions

Signal	Source	Width	Description
PCLK	Clock	1	**PCLK** is a clock signal. All APB signals are timed against the rising edge of **PCLK**.
PRESETn	System bus reset	1	**PRESETn** is the reset signal and is active-LOW. **PRESETn** is normally connected directly to the system bus reset signal.
PADDR	Requester	ADDR_WIDTH	**PADDR** is the APB address bus. **PADDR** can be up to 32 bits wide.
PSELx	Requester	1	The Requester generates a **PSELx** signal for each Completer. **PSELx** indicates that the Completer is selected and that a data transfer is required.
PENABLE	Requester	1	**PENABLE** indicates the second and subsequent cycles of an APB transfer.

2 Signal Descriptions

Signal	Source	Width	Description
PWRITE	Requester	1	**PWRITE** indicates an APB write access when HIGH and an APB read access when LOW.
PWDATA	Requester	DATA_ WIDTH	The **PWDATA** write data bus is driven by the APB bridge Requester during write cycles when **PWRITE** is HIGH. **PWDATA** can be 8, 16, or 32 bits wide.
PREADY	Completer	1	**PREADY** is used to extend an APB transfer by the Completer.
PRDATA	Completer	DATA_ WIDTH	The **PRDATA** read data bus is driven by the selected Completer during read cycles when **PWRITE** is LOW. **PRDATA** can be 8, 16, or 32 bits wide.
PSLVERR	Completer	1	**PSLVERR** is an optional signal that can be asserted HIGH by the Completer to indicate an error condition on an APB transfer.

2 Signal Descriptions

Address Bus:

- An APB interface has a single address bus, PADDR, for read and write transfers.
- PADDR indicates a byte address.

Data Bus:

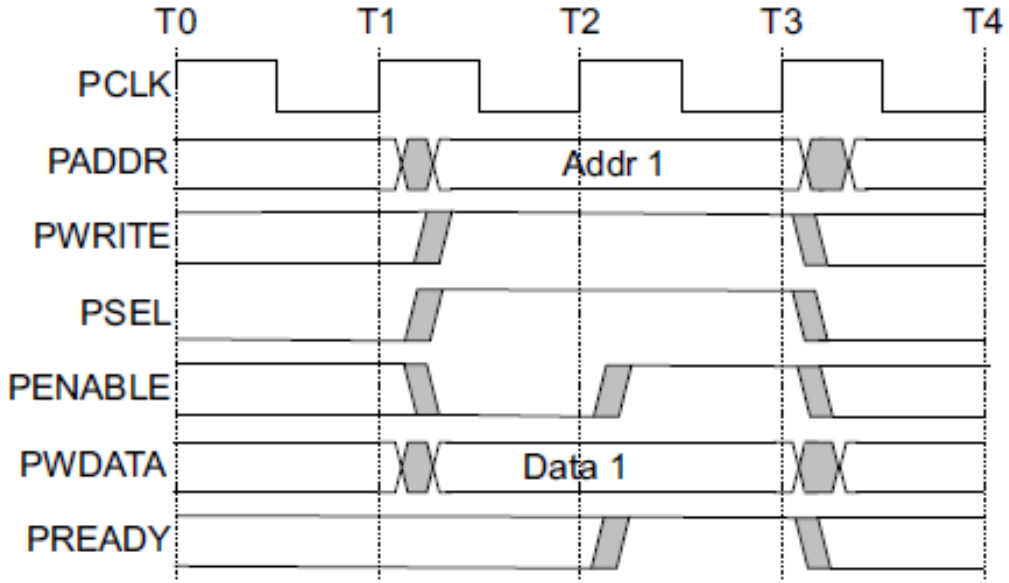
- The APB protocol has two independent data buses, one for read data and one for write data.
- The buses can be 8, 16, or 32 bits wide. The read and write data buses must have the same width.
- Data transfers cannot occur concurrently because the read data and write data buses do not have their own individual handshake signals.

3 Transfers

3 Transfers

Write Transfer With no wait states

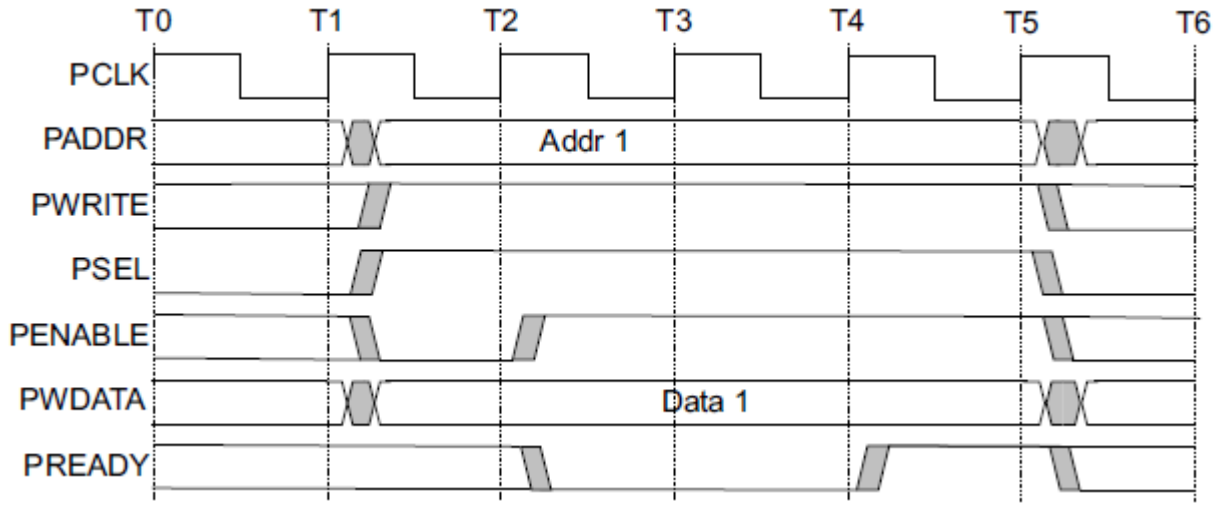
- At T1, the Setup phase of the write transfer occurs. The select signal, **PSEL**, is asserted, which means that **PADDR**, **PWRITE** and **PWDATA** must be valid.
- At T2, the Access phase of the write transfer is occurred where **PENABLE** is asserted. **PREADY** is asserted by the Completer at the rising edge of **PCLK** to indicate that the write data will be accepted at T3.
  PADDR, **PWDATA**, and any other control signals, must be stable until the transfer completes.
- At the end of the transfer, **PENABLE** is deasserted. **PSEL** is also deasserted, unless there is another transfer to the same peripheral.



3 Transfers

Write Transfer With wait states

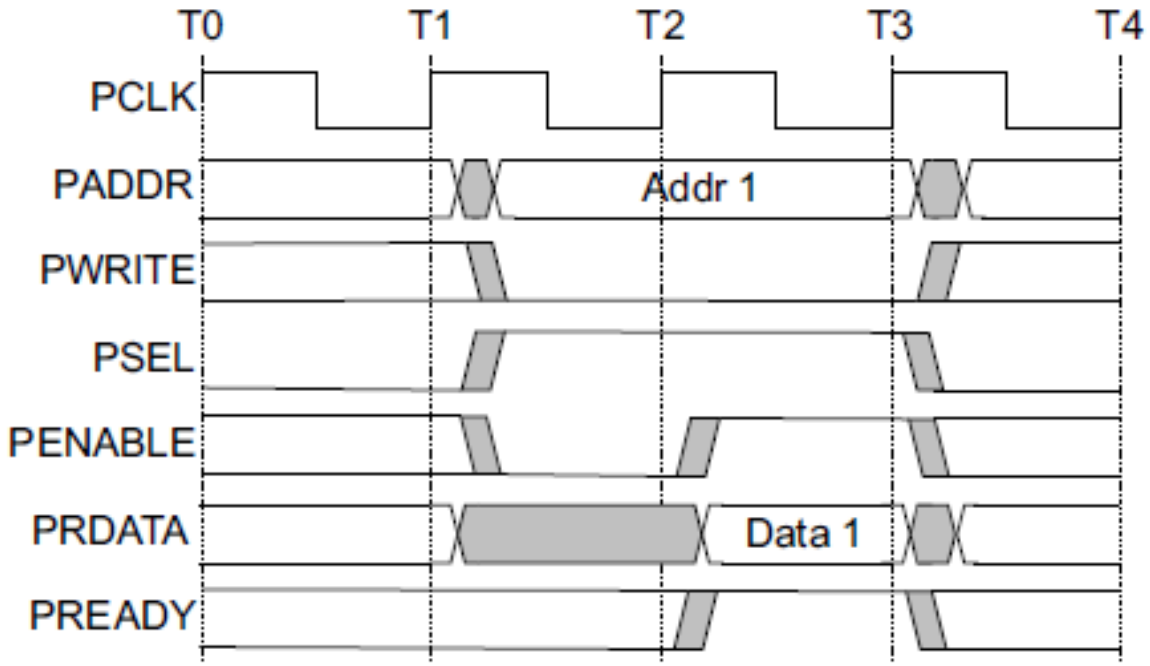
- During an Access phase, when **PENABLE** is HIGH, the Completer extends the transfer by driving **PREADY** LOW.
- The following signals remain unchanged while **PREADY** remains LOW:
  - Address signal, **PADDR**
  - Direction signal, **PWRITE**
  - Select signal, **PSELx**
  - Enable signal, **PENABLE**
  - Write data signal, **PWDATA**
- **PREADY** can take any value when **PENABLE** is LOW. This ensures that peripherals that have a fixed two cycle access can tie **PREADY** HIGH.



3 Transfers

Read Transfer with no wait states

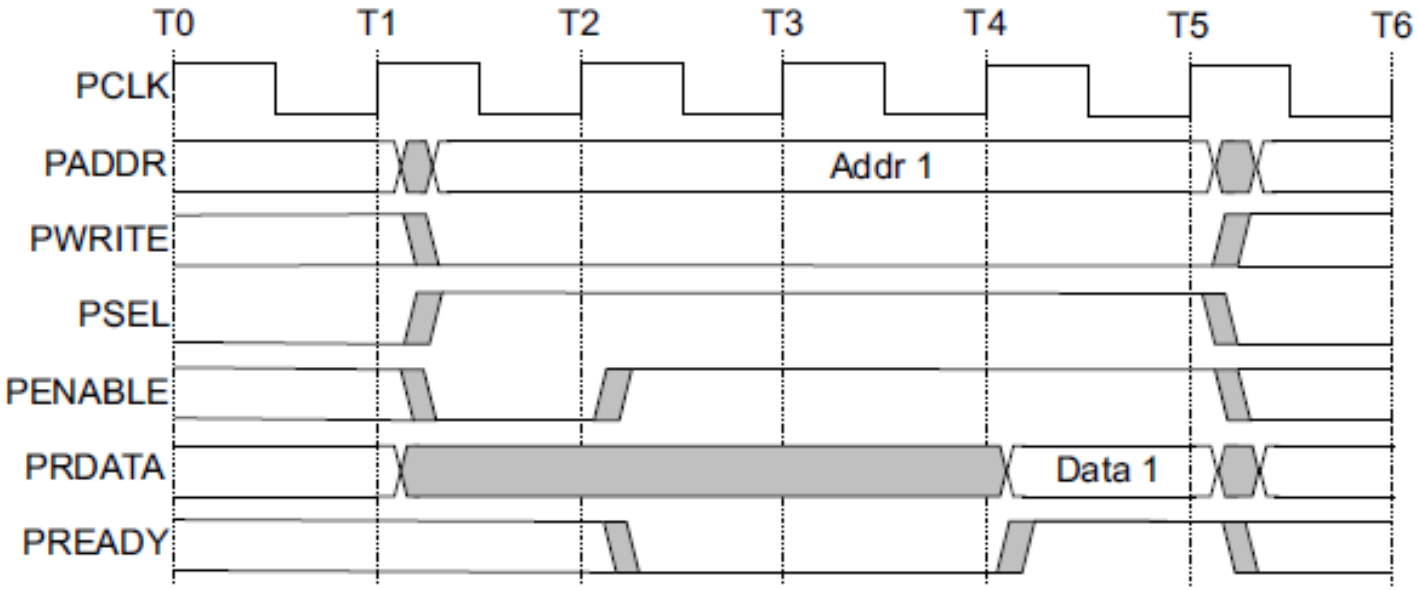
- The timing of the address, **PADDR**, write, **PWRITE**, select, **PSEL**, and enable, **PENABLE**, signals are the same as described in the write transfer.
- The Completer must provide the data before the end of the read transfer.



3 Transfers

Read Transfer with wait states

- The transfer is extended if **PREADY** is driven LOW during an Access phase. The following signals remain unchanged while **PREADY** remains LOW:
  - Address signal, **PADDR**
  - Direction signal, **PWRITE**
  - Select signal, **PSEL**
  - Enable signal, **PENABLE**
- Two cycles are added using **PREADY**. However, any number of additional cycles can be added, from zero upwards.

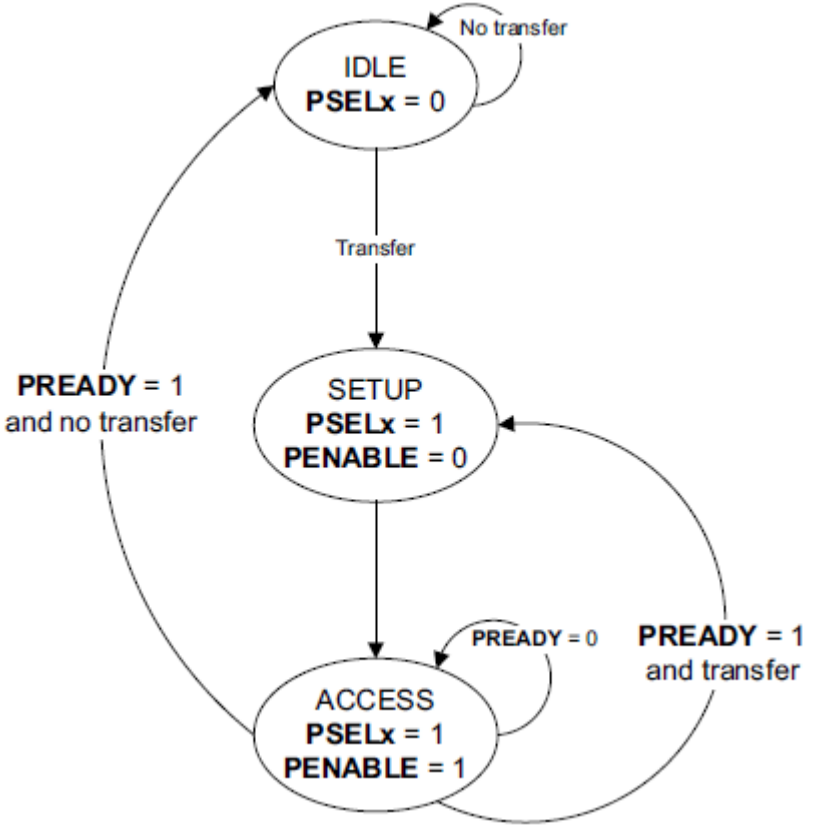


4 Operating States

4 Operating States

The state machine operates through the following states:

- **IDLE:** The default state of the APB interface.
- **SETUP:** When a transfer is required, the interface moves into the SETUP state, where the appropriate select signal, **PSELx**, is asserted. The interface only remains in the SETUP state for one clock cycle and always moves to the ACCESS state on the next rising edge of the clock.
- **ACCESS:** The enable signal, **PENABLE**, is asserted in the ACCESS state. The signals **(PADDR, PWRITE, PWDATA, PPROT)** must not change in the transition between SETUP and ACCESS and between cycles in ACCESS state.
  - If **PREADY** is held LOW, the interface remains in the ACCESS state.
  - If **PREADY** is driven HIGH, the ACCESS state is exited and the bus returns to the IDLE state if no more transfers are required or returns to the SETUP state if another transfer follows.



This presentation is prepared from:

Arm® Cortex®-M System Design Kit
Revision: r1p1
Technical Reference Manual.

Agenda

Cortex®-M System Design Kit

- # 3 Basic AHB-Lite components
  - # 3.6 AHB to APB sync-down bridge
- # 4 APB components
  - # 4.2 APB timer
  - # 4.3 APB UART

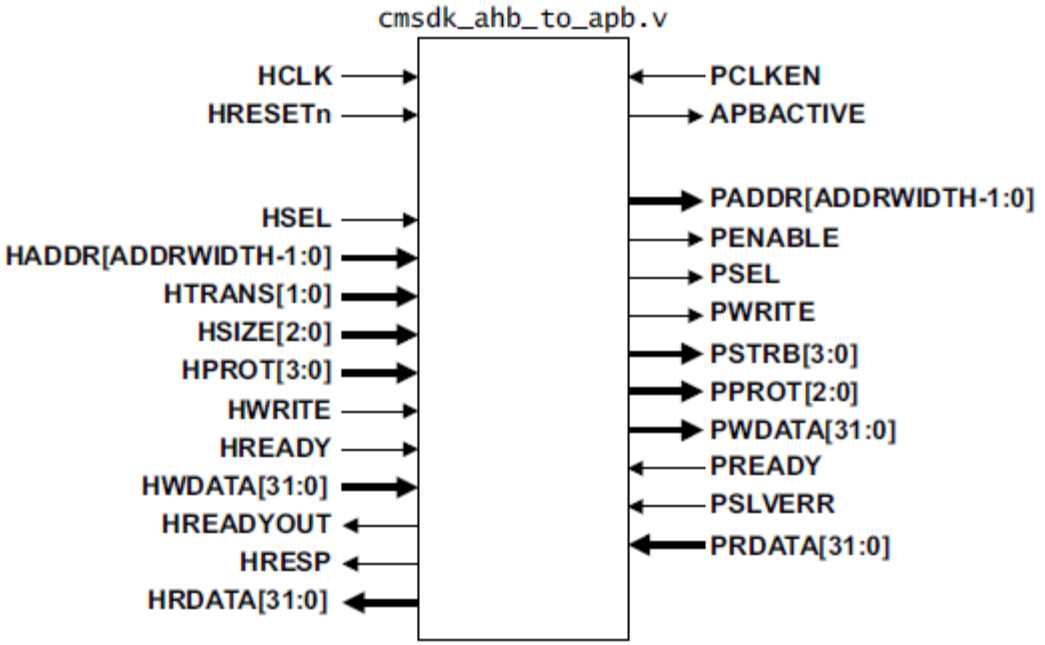
Agenda

Cortex®-M System Design Kit

- # 3 Basic AHB-Lite components
  - # 3.6 AHB to APB sync-down bridge
- # 4 APB components
  - # 4.2 APB timer
  - # 4.3 APB UART

3.6 AHB to APB sync-down bridge

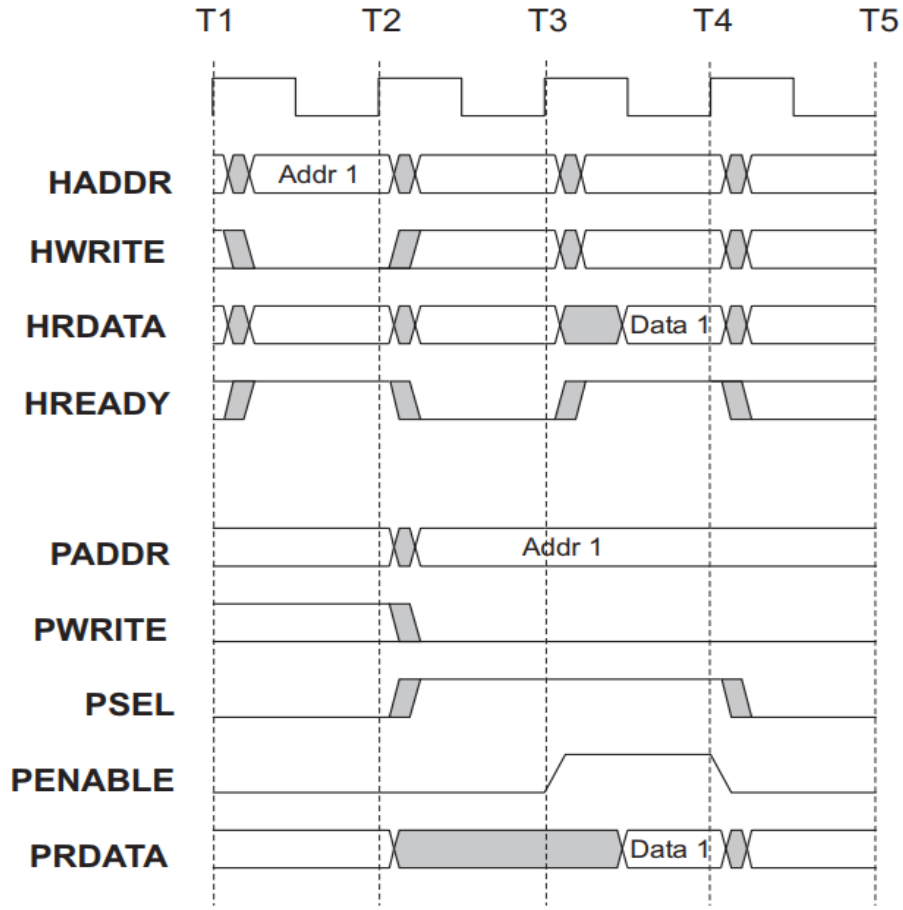
- The AHB to APB sync-down bridge, cmsdk_ahb_to_apb.v.
- The AHB to APB bridge has an output that is called **APBACTIVE** that controls the clock gating cell for generation of a gated **PCLK**.
  The gated **PCLK** is called **PCLKG** in the example system.
- When there is no APB transfer, this signal is LOW and stops **PCLKG**. Peripherals that are designed with separate clock pins for bus logic and peripheral operation can use the gated **PCLK** to reduce power consumption.
- This block requires an APB clock that is synchronized to **HCLK**. **PCLK** can be divided or the same as **HCLK** by using **PCLKEN**.
- When developing a system for AMBA 2.0, you can tie **PSLVERR** LOW, and **PREADY** HIGH.



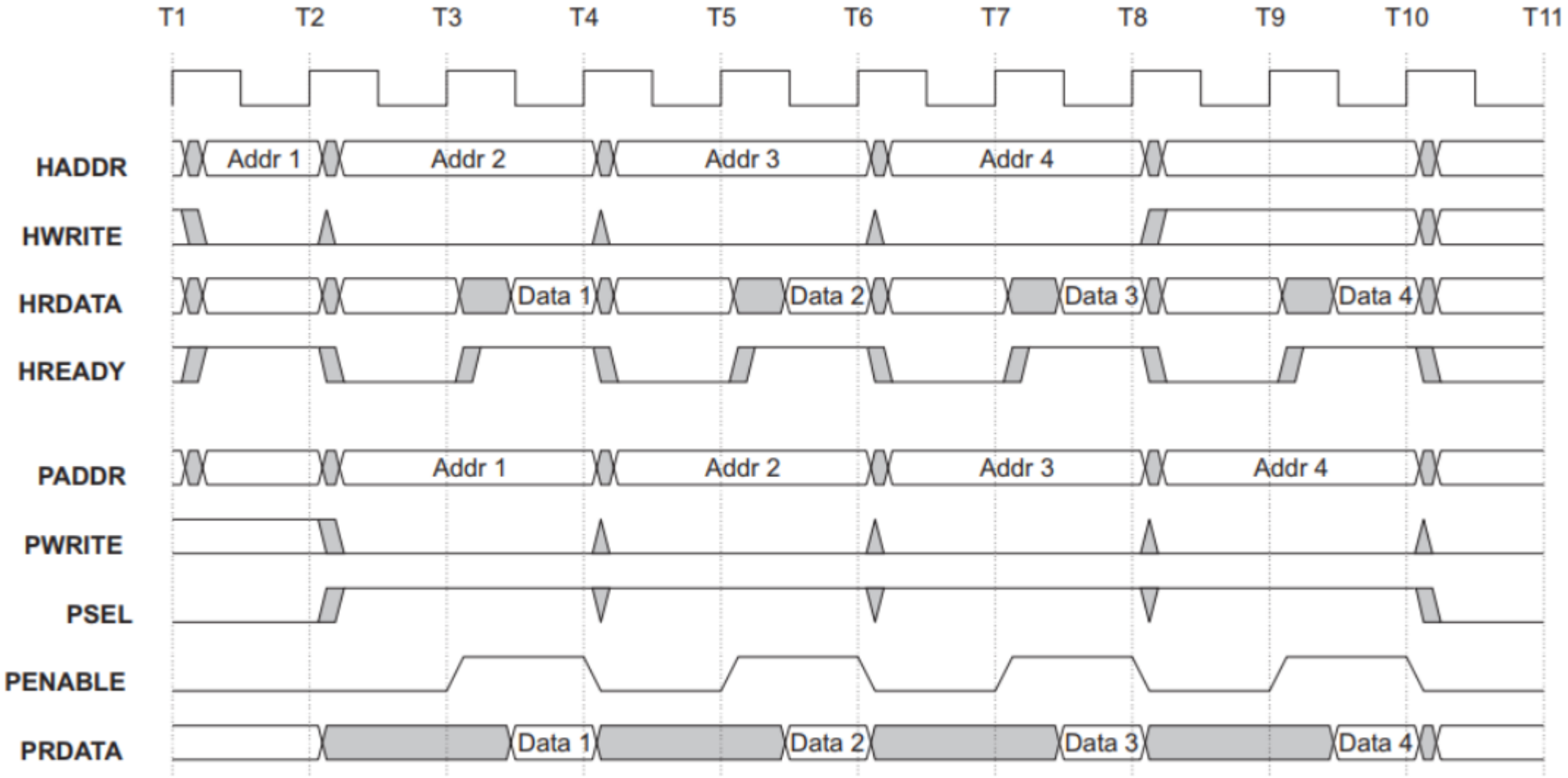
3.6 AHB to APB sync-down bridge

For Read Transfer:

- Transfer starts on the AHB bus at T1 and address is sampled by APB Bridge at T2.
- If transfer is for peripheral bus, address is broadcasted on the bus and appropriate PSELx is generated.
- The first cycle on peripheral bus is SETUP cycle, followed by ENABLE cycle, when PENABLE is asserted.
- During the ENABLE cycle the peripheral must provide read data.
- Normally the read data will be routed to AHB directly at the end of ENABLE cycle, where the AHB master samples the data at rising edge of clock.
- In very high clock systems, it is necessary for the bridge to register the read data at the end of enable cycle and then drive back to AHB Bridge in next clock cycle



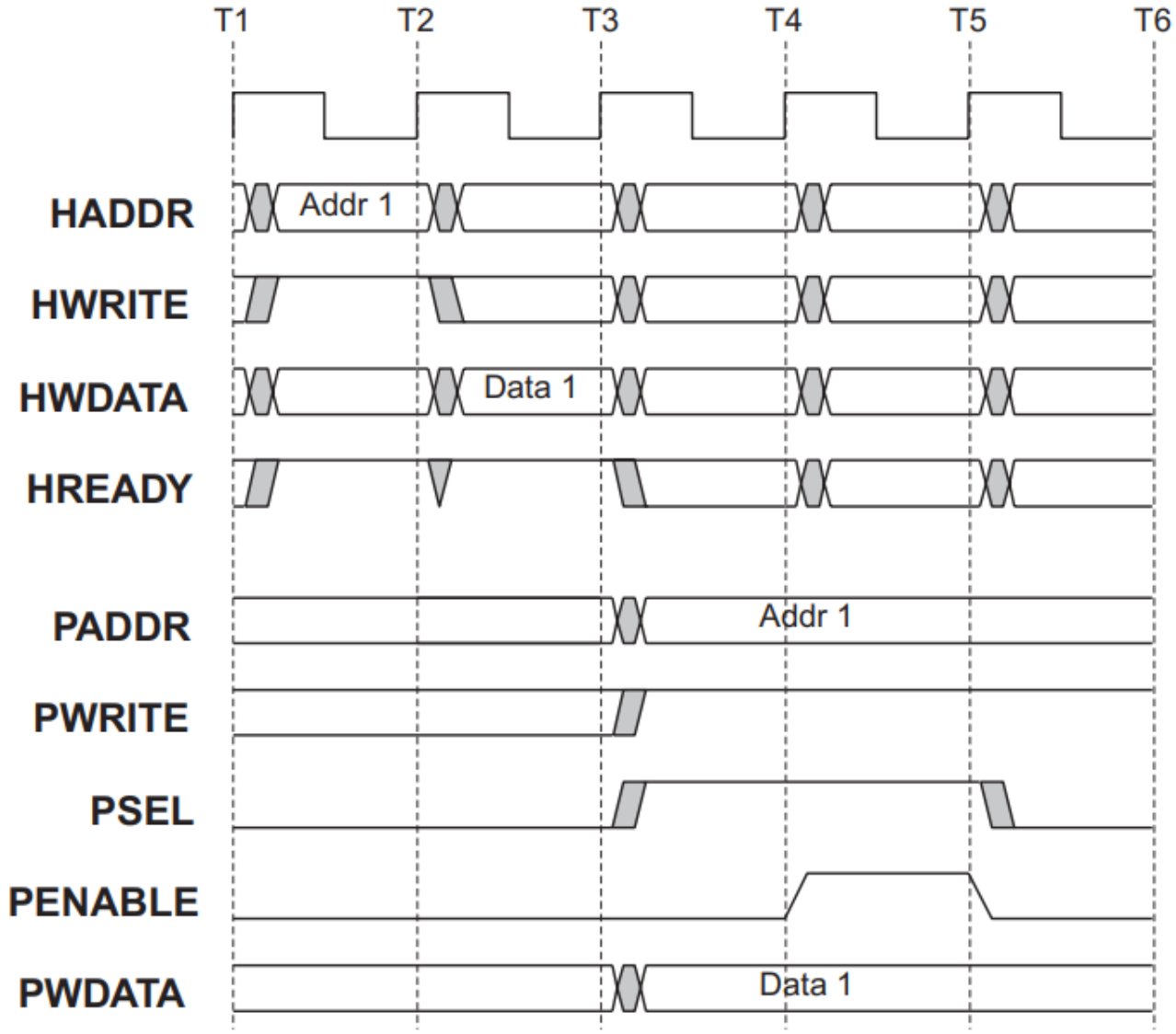
3.6 AHB to APB sync-down bridge



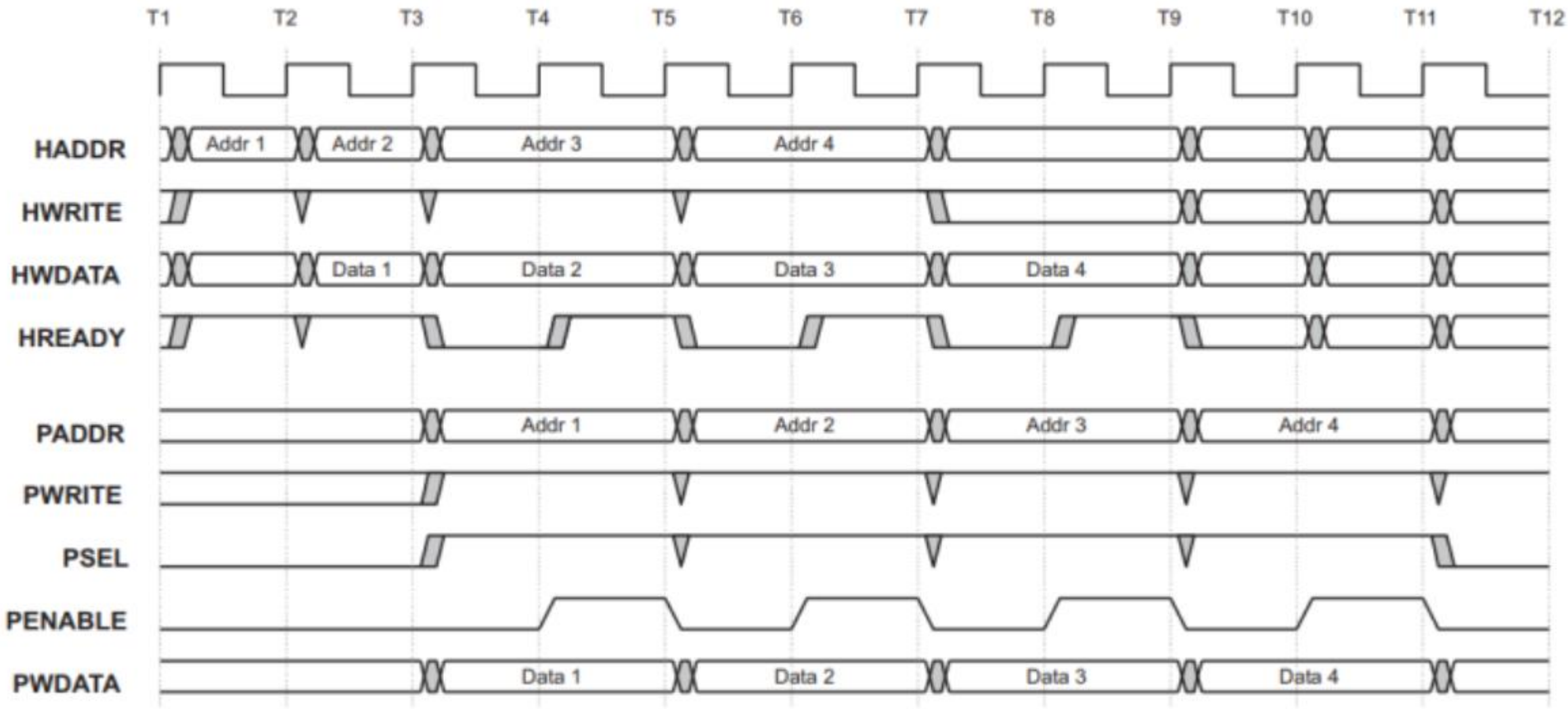
3.6 AHB to APB sync-down bridge

For Write Transfer:

- Single write transfer to APB can occur with zero wait states
- Bridge is responsible for sampling address and data of transfer and holding these values for the duration of APB write transfer.
- While first transfer completes with zero wait states, subsequent transfers to the bus will require a single wait state for each transfer performed.
- It is necessary for the bridge to contain two address registers, in order that the bridge can sample the address of the next transfer while current transfer continues on the peripheral bus.



3.6 AHB to APB sync-down bridge



Agenda

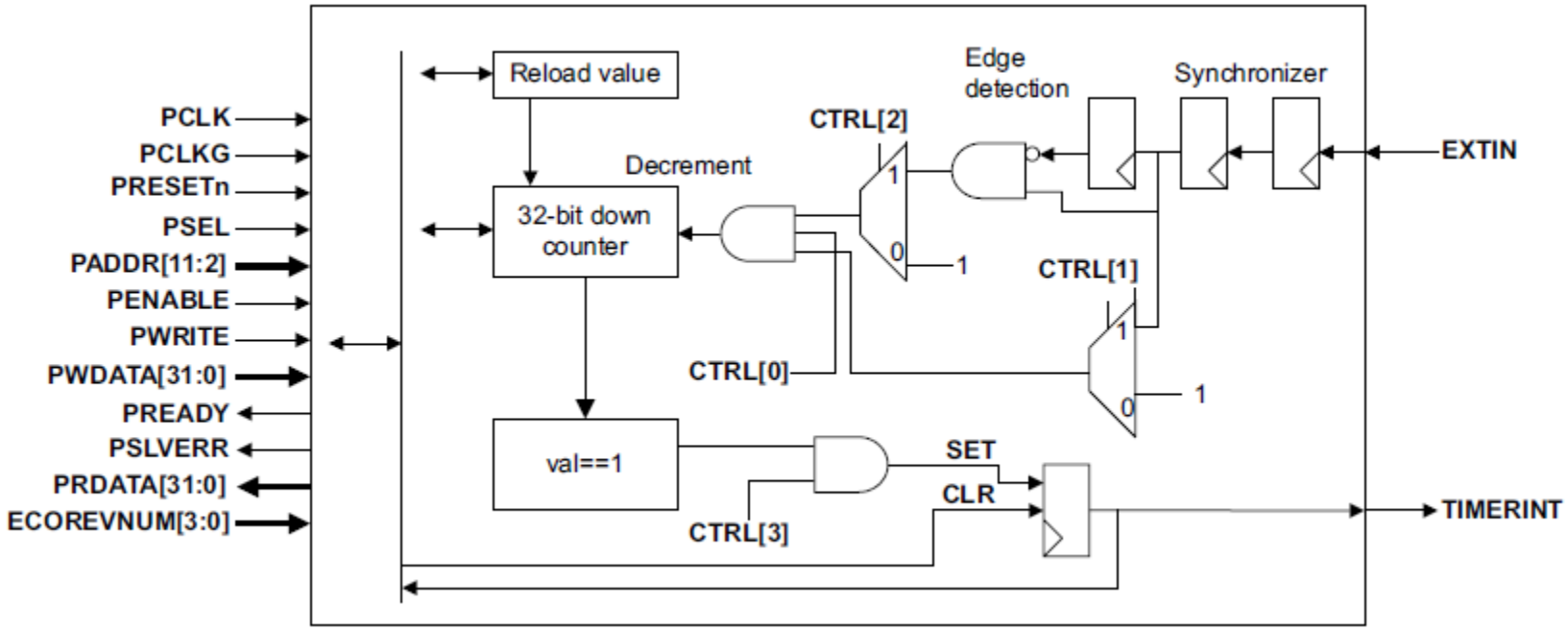
Cortex®-M System Design Kit

- # 3 Basic AHB-Lite components
  - # 3.6 AHB to APB sync-down bridge
- # 4 APB components
  - # 4.2 APB timer
  - # 4.3 APB UART

4.2 APB timer

The APB timer, cmsdk_apb_timer.v, is a 32-bit down-counter with the following features:

- You can generate an interrupt request signal, **TIMERINT**, when the counter reaches 0. The interrupt request is held until it is cleared by writing to the INTCLEAR register.
- You can use the external input signal, **EXTIN**, as a timer enable.
- If the APB timer count reaches 0 and, at the same time, the software clears a previous interrupt status, the interrupt status is set to 1.
- The external clock, **EXTIN**, must be slower than half of the peripheral clock because it is sampled by a double flip-flop and then goes through edge-detection logic when the external inputs act as a clock.



- A separate clock pin, **PCLKG** permits the clock to peripheral register logic to stop when there is no APB activity.

4.2 APB timer

- **3.5.2 Programmers model**

Name	Base offset	Type	Width	Reset	value Description
CTRL	0x000	RW	4	4 0x0	**[3]** Timer interrupt enable. **[2]** Select external input as clock. **[1]** Select external input as enable. **[0]** Enable.
VALUE	0x004	RW	32	0x00000000	**[31:0]** Current value.
RELOAD	0x008	RW	32	0x00000000	**[31:0]** Reload value. A write to this register sets the current value after it reaches 0.
INTSTATUS INTCLEAR	0x00C	RW	1	0x0	**[0]** Timer interrupt. Write one to clear.

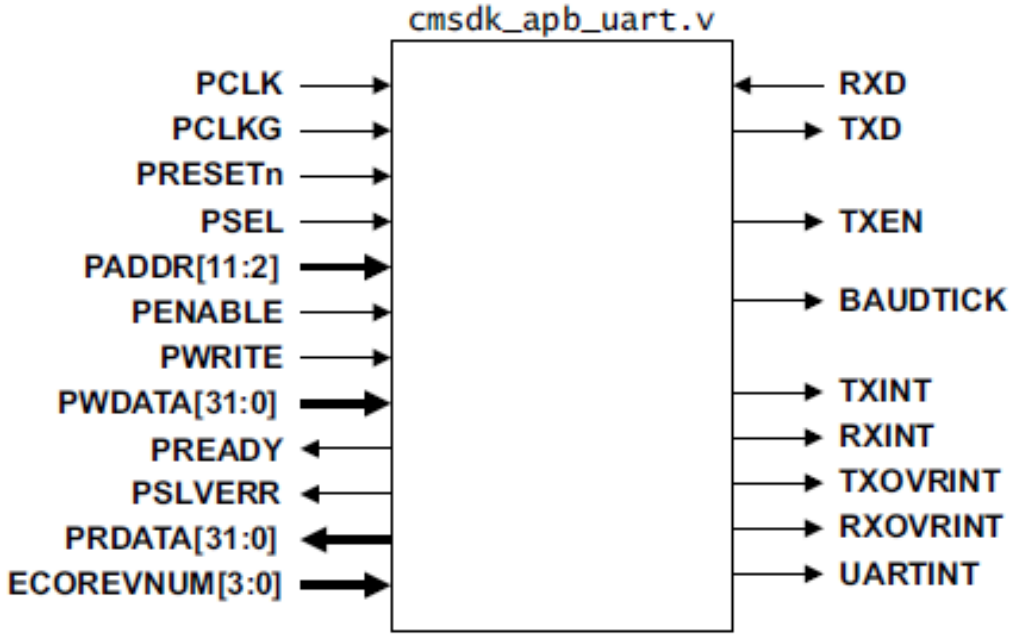
Agenda

Cortex®-M System Design Kit

- # 3 Basic AHB-Lite components
  - # 3.6 AHB to APB sync-down bridge
- # 4 APB components
  - # 4.2 APB timer
  - # 4.3 APB UART

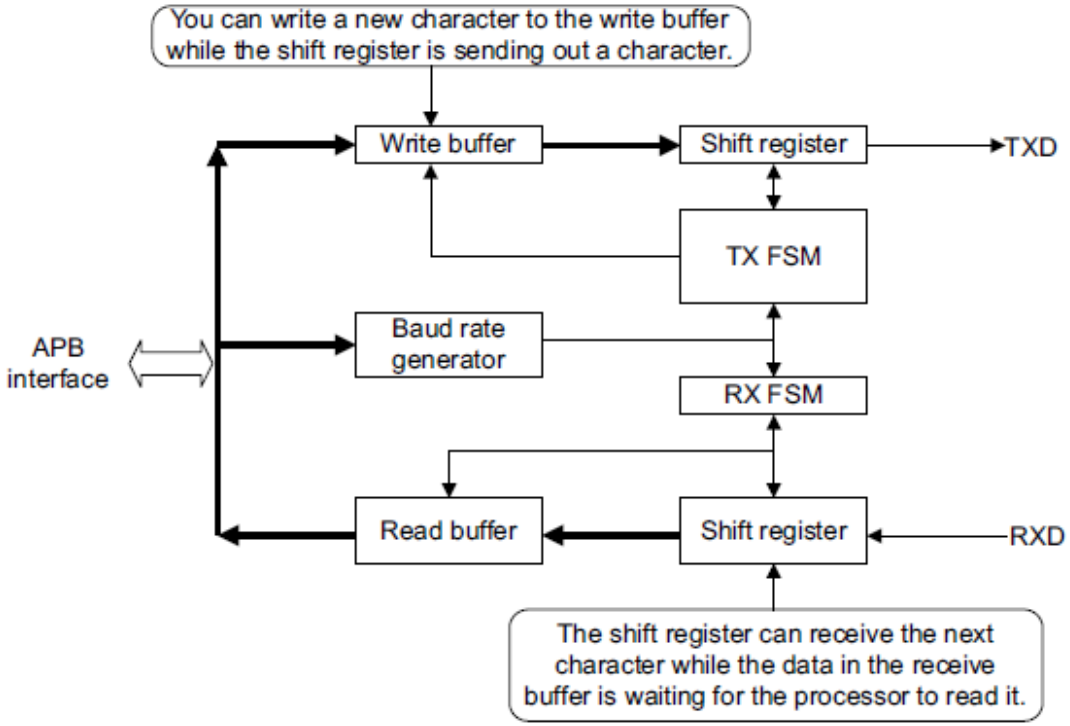
4.3 APB UART

- The APB UART, cmsdk_apb_uart.v, is a simple design that supports 8-bit communication without parity, and is fixed at one stop bit per configuration.



4.3 APB UART

- This buffer arrangement is sufficient for most simple embedded applications. For example, a processor running at 30MHz with a baud rate of 115200 means a character transfer every $30 \times 10^6 \times (1 + 8 + 1) / 115200 = 2604$ cycles.
- For duplex communication, the processor might receive an interrupt every 1300 clock cycles.
- Because the interrupt response time and the handler execution time are usually quite short, this leaves sufficient processing time for the thread.
- The design includes a double flip-flop synchronization logic for the receive data input.



4.3 APB UART

- **3.5.2 Programmers model**

Name	Base offset	Type	Width	Reset	value Description
DATA	0x000	RW	8	0x--	**[7:0]** Data value. **Read** Received data, or **Write** Transmit data.
STATE	0x004	RW	4	0x0	**[3]** RX buffer overrun, write 1 to clear. **[2]** TX buffer overrun, write 1 to clear. **[1]** RX buffer full, read-only. **[0]** TX buffer full, read-only.
CTRL	0x008	RW	7	0x00	**[6]** High-speed test mode for TX only. **[5]** RX overrun interrupt enable. **[4]** TX overrun interrupt enable. **[3]** RX interrupt enable. **[2]** TX interrupt enable. **[1]** RX enable. **[0]** TX enable.
INTSTATUS INTCLEAR	0x00C	RW	4	0x0	**[3]** RX overrun interrupt. Write 1 to clear. **[2]** TX overrun interrupt. Write 1 to clear. **[1]** RX interrupt. Write 1 to clear. **[0]** TX interrupt. Write 1 to clear.
BAUDDIV	0x010	RW	20	0x00000	**[19:0]** Baud rate divider. The minimum number is 16.

4.3 APB UART

- The APB UART supports a high-speed test mode, useful for simulation during SoC or ASIC development. When CTRL[6] is set to 1, the serial data is transmitted at one bit per clock cycle.
  You can remove this feature for silicon products to reduce the gate count by removing bit[6] of the **reg_ctrl** signal.
- The APB interface always sends an OKAY response with no wait state and is two cycles per transfer.
- You must program the baud rate divider register before enabling the UART. For example, if the **PCLK** is running at 12MHz, and the required baud rate is 9600, program the baud rate divider register as $12 \times 10^6 / 9600 = 1250$.
- The **BAUDTICK** output pulses at a frequency of 16 times that of the programmed baud rate.
  You can use this signal for capturing UART data in a simulation environment.
- The **TXEN** output signal indicates the status of CTRL[0]. You can use this signal to switch a bidirectional I/O pin in a silicon device to UART data output mode automatically when the UART transmission feature is enabled.
- The buffer overrun status in the STATE field is used to drive the overrun interrupt signals.
- Therefore, clearing the buffer overrun status deasserts the overrun interrupt, and clearing the overrun interrupt bit also clears the buffer overrun status bit in the STATE field.

This presentation is prepared from:

STMicroelectronics® RM0493.

Agenda

STM32WB Architecture

- # 1 Introduction
- # 2 System Architecture
- # 3 General-Purpose I/O
- # 4 Serial I/O
- # 5 Timers
- # 6 Interrupts
- # 7 Analog I/O

Agenda

STM32WB Architecture

1 Introduction

2 System Architecture

3 General-Purpose I/O

4 Serial I/O

5 Timers

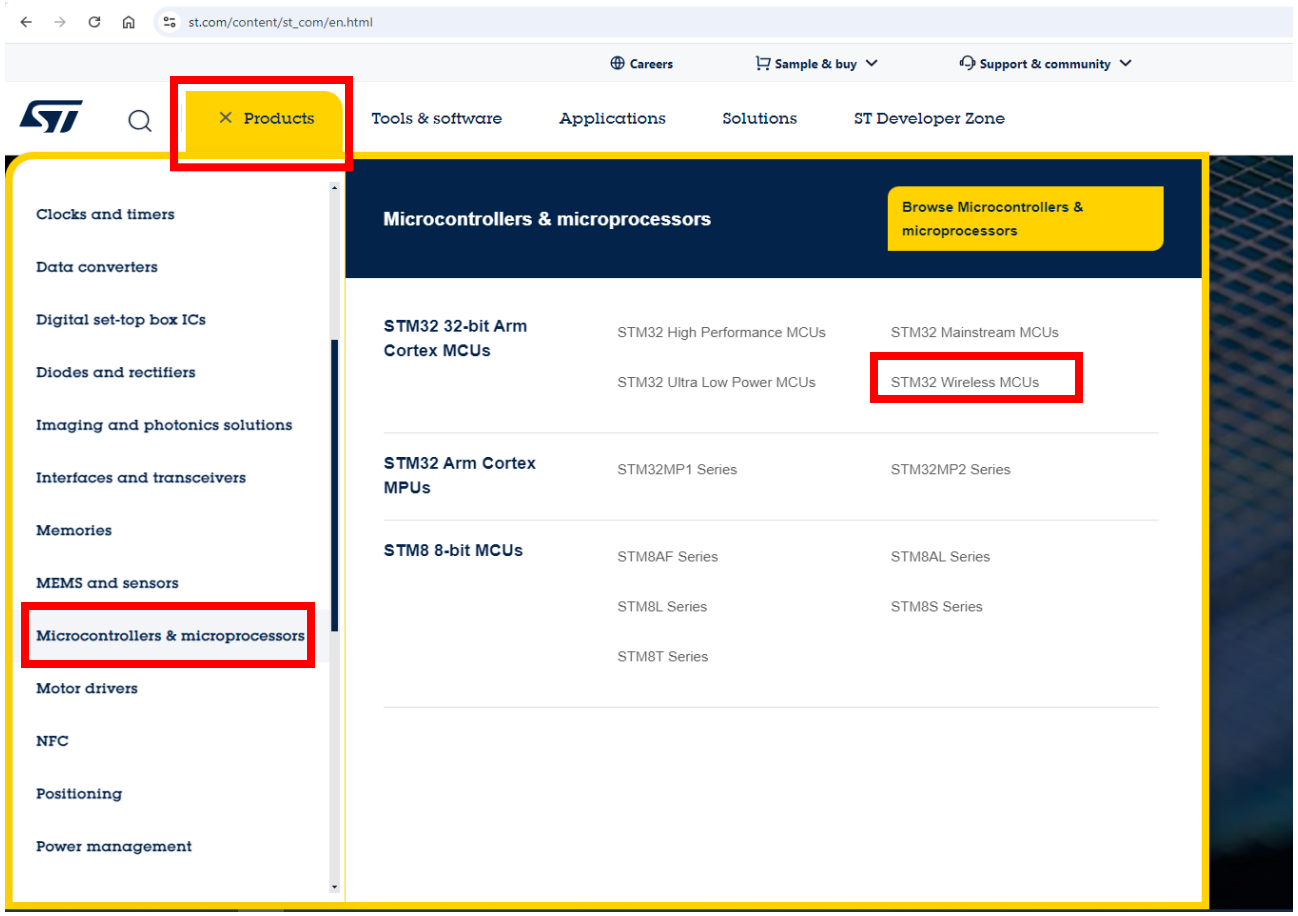
6 Interrupts

7 Analog I/O

1 Introduction

2 Embedded I/O Systems

- We will present **STM32WBAxxx** as an example for the microcontroller with embedded IO system.
  - Go to https://www.st.com page.
  - From “Products” tab, select “Microcontrollers & Microprocessors”.
  - Then choose “STM32 Wireless MCUs”.
  - New page will open, with another tab, select “Documentation”.
  - Do a filter for “Reference Manual”.
  - Finally, select “RM0493” book.



Agenda

STM32WB Architecture

1 Introduction

2 System Architecture

3 General-Purpose I/O

4 Serial I/O

5 Timers

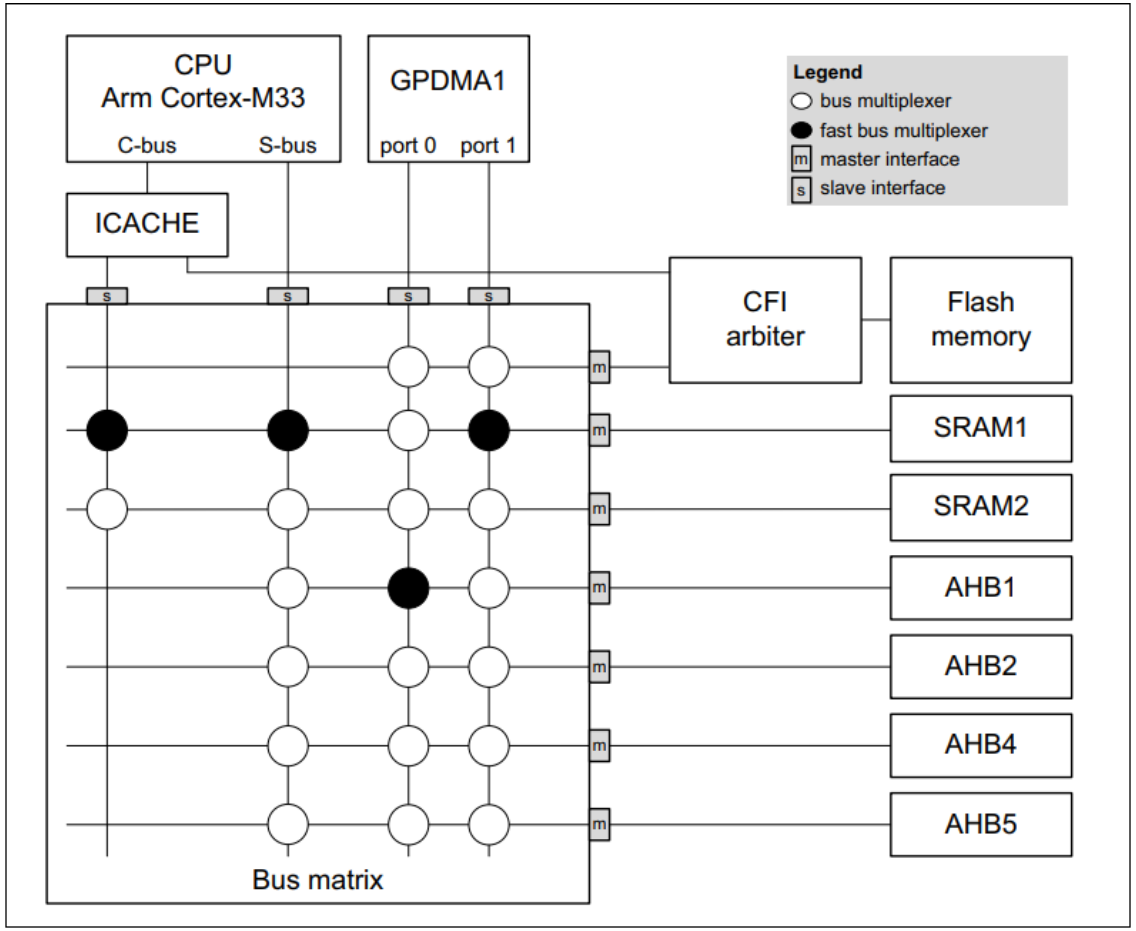
6 Interrupts

7 Analog I/O

2 System Architecture

2 System Architecture

- A 32-bit multilayer AHB bus matrix interconnects:
- Masters:
  - CPU core C-bus (128-bit) ICACHE connecting to flash memory.
  - CPU core C-bus (32-bit) ICACHE connecting to SRAM.
  - CPU core S-bus.
  - GPDMA1 port 0.
  - GPDMA1 port 1.
- Slaves:
  - Internal flash memory on the CPU core C-bus.
  - Internal flash memory on the GPDMA1 bus.
  - Internal SRAM1, SRAM2.
  - AHB1 (including APB1 and APB2), AHB2, AHB4 (including APB7) and AHB5 peripherals.



2.1.1 CPU C-bus

This bus is used for instruction fetch and data access to the internal memories mapped in code.
This bus targets the internal flash memory and the internal SRAMs via the ICACHE decoder.

2.1.2 CPU S-bus

This bus connects the system bus of the CPU to the bus matrix, and it is used by the core to access data located in a peripheral or SRAM area.
This bus targets the internal SRAM (SRAM1 and SRAM2), the AHB and APB peripherals.

2.1.3 GPDMA1-bus

The buses connect the two AHB master interfaces of the GPDMA1 to the bus matrix.
This targets the internal flash memory, the internal SRAMs, the AHB and APB peripherals.

2.1.4 Bus matrix

The bus matrix manages the access arbitration (based on a round-robin algorithm) between masters and features a fast bus multiplexer used to connect each master to a given slave without latency. The slaves behind a standard bus multiplexer suffer an access latency of at least one cycle.

2.1.5 AHB/APB bridges

The three bridges AHB1 to APB1, AHB1 to ABP2, and AHB4 to APB7 provide full synchronous connections between the AHB and the APB buses.

After each device reset, the clock of peripherals having an EN bit in the RCC is disabled.

Before using a peripheral, its clock must be enabled in the **RCC_AHBxENR** and **RCC_APBxENR**.

2 System Architecture

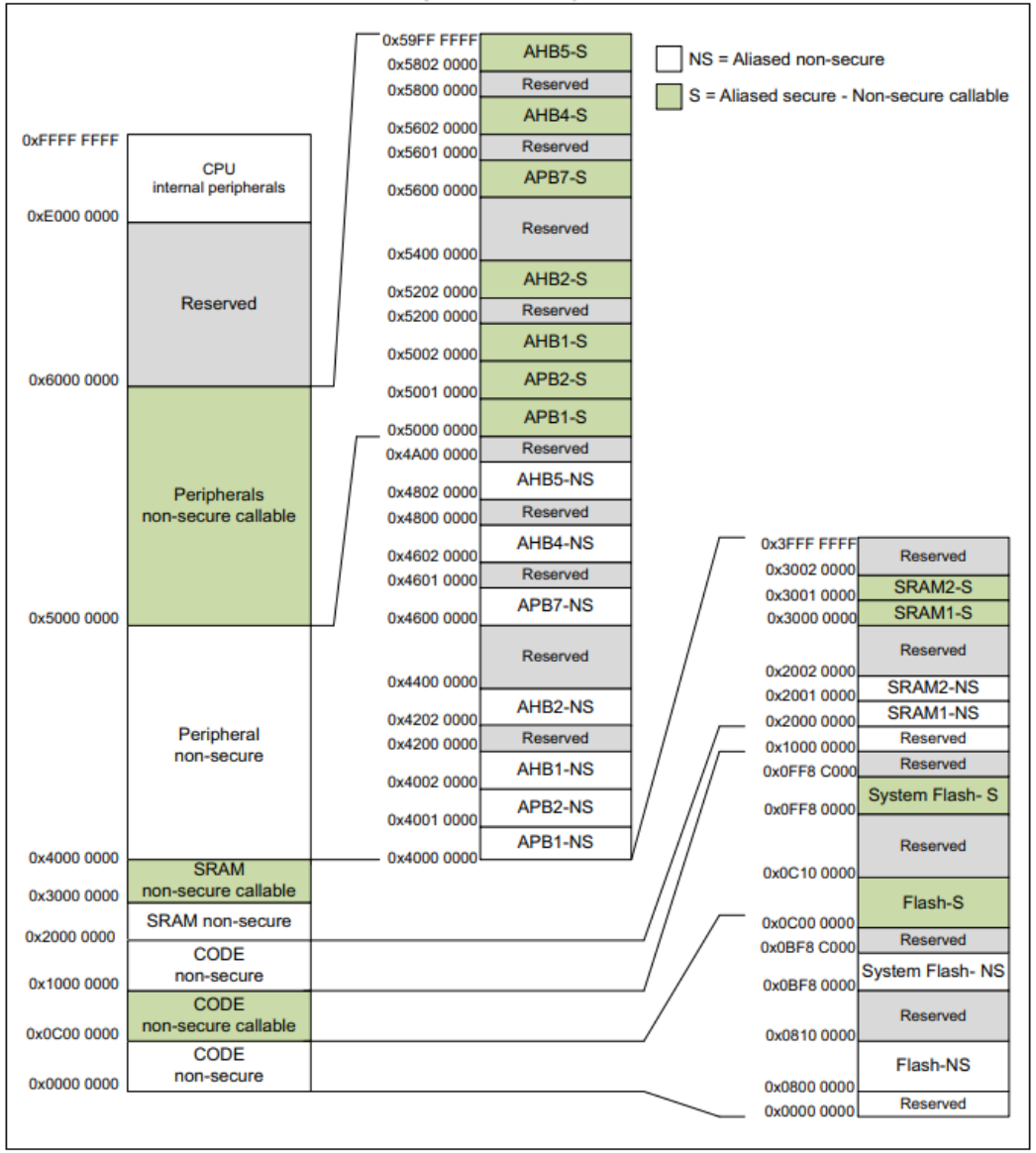
2.2.1 Introduction

Program memory, data memory, registers and I/O ports are organized within the same linear 4-Gbyte address space.

The bytes are coded in memory in **Little Endian** format.

The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte the most significant.

The addressable memory space is divided into eight main blocks, of 512 Mbytes each (2^{28}Byte).



2.2.2 Memory map and register boundary addresses

AHB1			AHB2	AHB4		AHB5
GPDMA1	APB1	APB2	GPIO	PWR	APB7	RXTXRAM
CRC	USART2	USART1	RNG	RCC	LPUART1	SEQRAM
ICACHE	I2C1	SPI1	AES	EXTI	I2C3	RADIO
FLASH itf	LPTIM2	TIM1	SAES	ADC4	SPI3	
RAMCFG	TIM2	TIM16	PKA		LPTIM1	
GTZC	TIM3	TIM17	HASH		RTC	
TSC	IWDG		HSEM		TAMP	
	WWDG				SYSCFG	

2.2.2 Memory map and register boundary addresses

Bus	Non-secure callable boundary address(1)	Non-secure boundary address(1)	Size (bytes)	Peripheral
-	0x5A00 0000 - 0xDFFF FFFF	0x4A00 0000 - 0x4FFF FFFF	-	Reserved
AHB5	0x5802 C000 - 0x59FF FFFF	0x4802 C000- 0x49FF FFFF	-	Reserved
	0x5802 8000 - 0x5802 BFFF	0x4802 8000 - 0x4802 BFFF	16 K	RXTXRAM
	0x5802 1200 - 0x5802 7FFF	0x4802 1200 - 0x4802 7FFF	-	Reserved
	0x5802 1000 - 0x5802 11FF	0x4802 1000 - 0x4802 11FF	0.5 K	SEQRAM
	0x5802 0000 - 0x5802 0FFF	0x4802 0000 - 0x4802 0FFF	4 K	2.4 GHz RADIO

2.2.2 Memory map and register boundary addresses

Bus	Non-secure callable boundary address(1)	Non-secure boundary address(1)	Size (bytes)	Peripheral
-	0x5800 0000 - 0x5801 FFFF	0x4800 0000 - 0x4801 FFFF	-	Reserved
AHB4	0x5602 2400 - 0x57FF FFFF	0x4602 2400 - 0x47FF FFFF	-	Reserved
	0x5602 2000 - 0x5602 23FF	0x4602 2000 - 0x4602 23FF	1 K	EXTI
	0x5602 1400 - 0x5602 1FFF	0x4602 1400 - 0x4602 1FFF	-	Reserved
	0x5602 1000 - 0x5602 13FF	0x4602 1000 - 0x4602 13FF	1 K	ADC4
	0x5602 0C00 - 0x5602 0FFF	0x4602 0C00 - 0x4602 0FFF	1 K	RCC
	0x5602 0800 - 0x5602 0BFF	0x4602 0800 - 0x4602 0BFF	1 K	PWR

2.2.2 Memory map and register boundary addresses

Bus	Non-secure callable boundary address(1)	Non-secure boundary address(1)	Size (bytes)	Peripheral
-	0x5601 0000 - 0x5601 FFFF	0x4601 0000 - 0x4601 FFFF	-	Reserved
APB7	0x5600 8000 - 0x5600 FFFF	0x4600 8000 - 0x4600 FFFF	-	Reserved
	0x5600 7C00 - 0x5600 7FFF	0x4600 7C00 - 0x4600 7FFF	1 K	TAMP
	0x5600 7800 - 0x5600 7BFF	0x4600 7800 - 0x4600 7BFF	1 K	RTC
	0x5600 4800 - 0x5600 77FF	0x4600 4800 - 0x4600 77FF	-	Reserved
	0x5600 4400 - 0x5600 47FF	0x4600 4400 - 0x4600 47FF	1 K	LPTIM1
	0x5600 2C00 - 0x5600 43FF	0x4600 2C00 - 0x4600 43FF	-	Reserved
	0x5600 2800 - 0x5600 2BFF	0x4600 2800 - 0x4600 2BFF	1 K	I2C3
	0x5600 2400 - 0x5600 27FF	0x4600 2400 - 0x4600 27FF	1 K	LPUART1
	0x5600 2000 - 0x5600 23FF	0x4600 2000 - 0x4600 23FF	1 K	SPI3
	0x5600 0800 - 0x5600 1FFF	0x4600 0800 - 0x4600 1FFF	-	Reserved
	0x5600 0400 - 0x5600 07FF	0x4600 0400 - 0x4600 07FF	1 K	SYSCFG
	0x5600 0000 - 0x5600 03FF	0x4600 0000 - 0x4600 03FF	-	Reserved

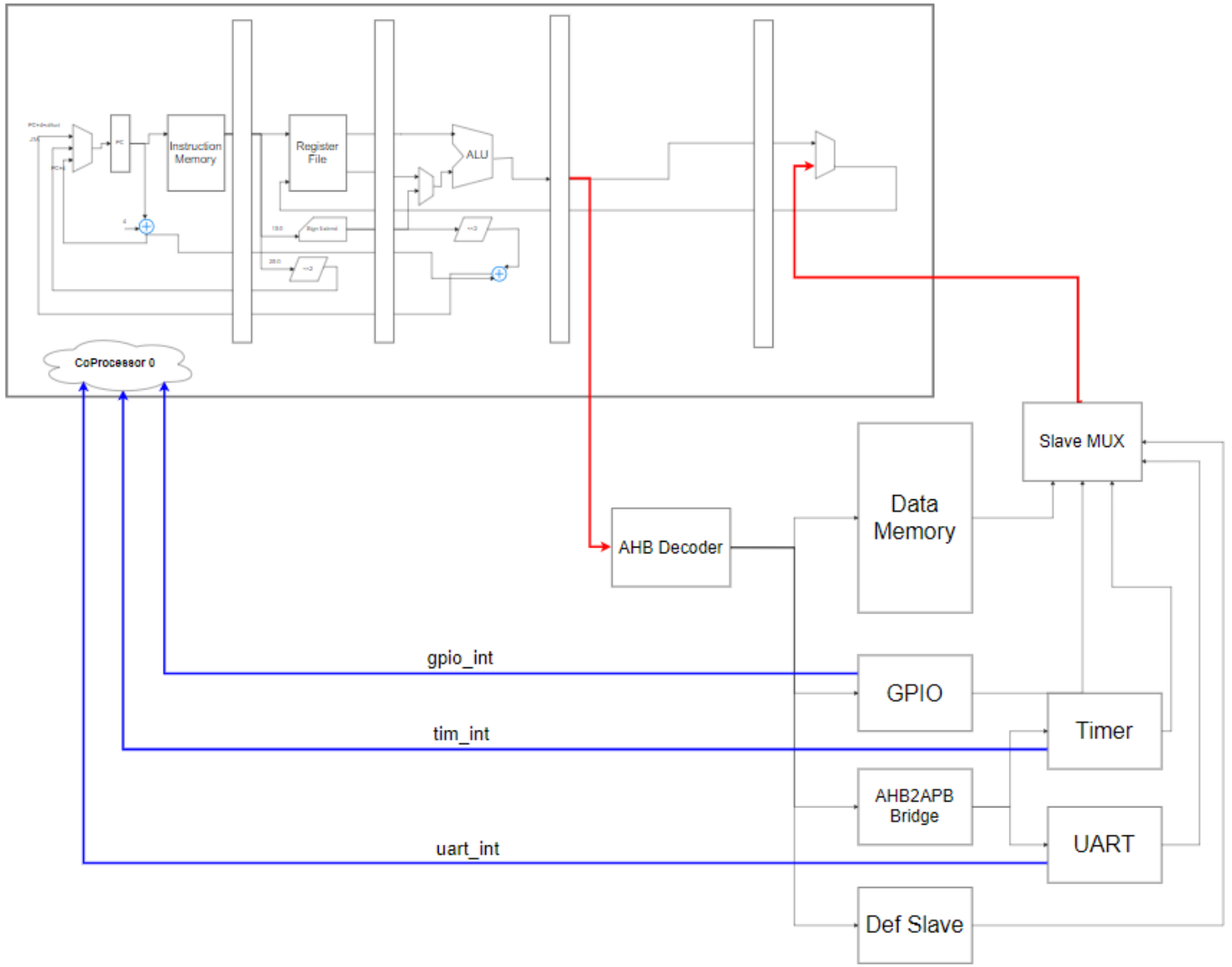
2.2.2 Memory map and register boundary addresses

Bus	Non-secure callable boundary address	Non-secure boundary address	Size (bytes)	Peripheral
-	0x5400 0000 - 0x55FF FFFF	0x4400 0000- 0x45FF FFFF	-	Reserved
AHB2	0x520C 4000 - 0x53FF FFFF	0x420C 4000 - 0x43FF FFFF	-	Reserved
	0x520C 2000 - 0x520C 3FFF	0x420C 3400 - 0x420C 3FFF	8 K	PKA
	0x520C 1C00 - 0x520C 1FFF	0x420C 1C00 - 0x420C 1FFF	1 K	HSEM
	0x520C 1000 - 0x520C 1BFF	0x420C 1000 - 0x420C 1BFF	-	Reserved
	0x520C 0C00 - 0x520C 0FFF	0x420C 0C00 - 0x420C 0FFF	1 K	SAES
	0x520C 0800 - 0x520C 0BFF	0x420C 0800 - 0x420C 0BFF	1 K	RNG
	0x520C 0400 - 0x520C 07FF	0x420C 0400 - 0x420C 07FF	1 K	HASH
	0x520C 0000 - 0x520C 03FF	0x420C 0000 - 0x420C 03FF	1 K	AES
	0x5202 2000 - 0x520B FFFF	0x4202 2000 - 0x420B FFFF	-	Reserved
	0x5202 1C00 - 0x5202 1FFF	0x4202 1C00 - 0x4202 1FFF	1 K	GPIOH
	0x5202 0C00 - 0x5202 1BFF	0x4202 0C00 - 0x4202 1BFF	-	Reserved
	0x5202 0800 - 0x5202 0BFF	0x4202 0800 - 0x4202 0BFF	1 K	GPIOC
	0x5202 0400 - 0x5202 07FF	0x4202 0400 - 0x4202 07FF	1 K	GPIOB
	0x5202 0000 - 0x5202 03FF	0x4202 0000 - 0x4202 03FF	1 K	GPIOA

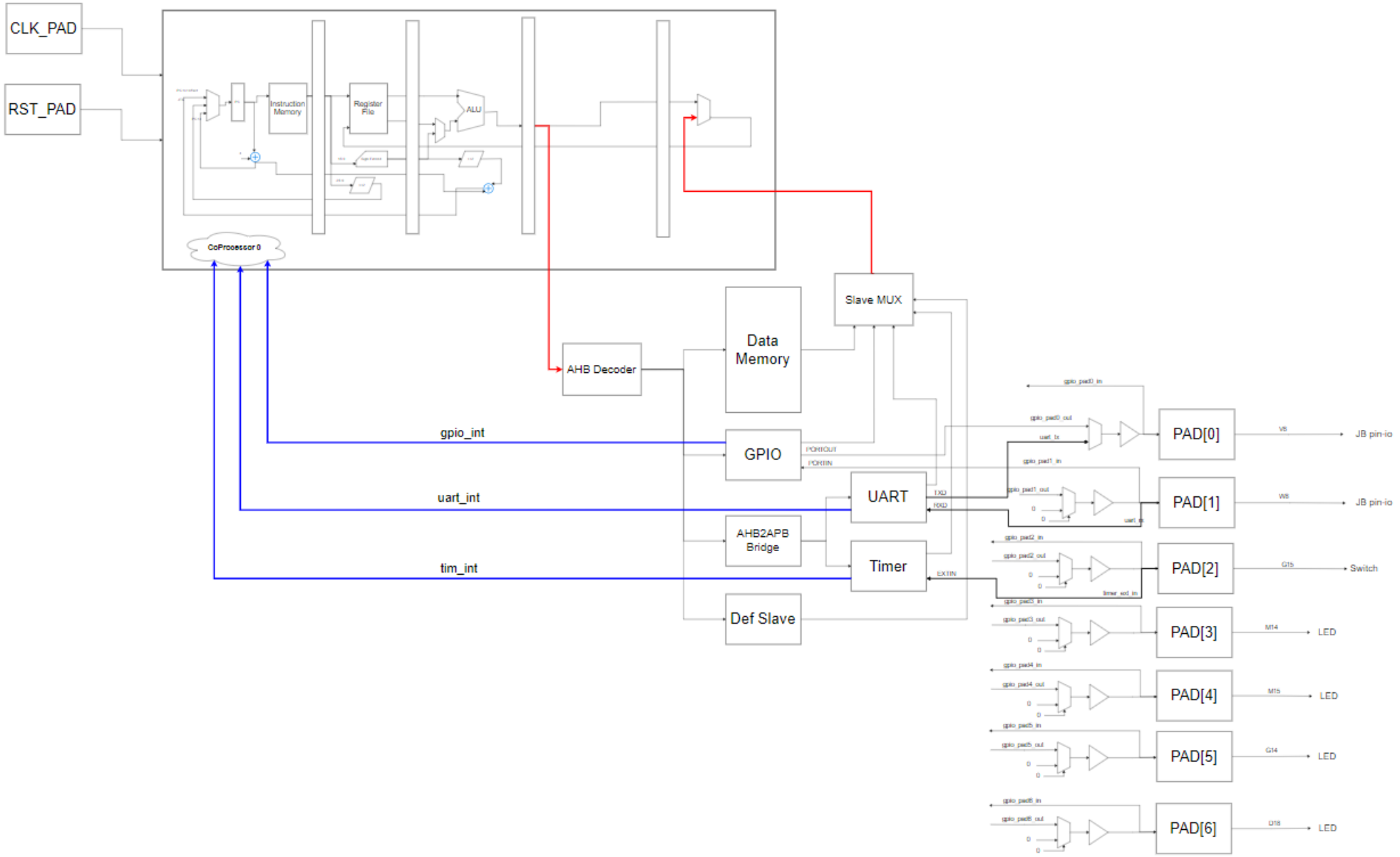
Assignment #8

Assignment #8

- Our target is to implement the shown architecture.



Assignment #8



Assignment #8

- Integrate *cmsdk_ahb_to_apb*, *cmsdk_apb_uart* and *cmsdk_apb_timer* IPs to our SoC with AHB interface and connect their interrupt back to co-processor 0.
- Do a presentation to explain your work (15 mins).
- Deadline: Friday, 20th Dec 2024, at 11:59 PM.
- Deliveries: Full design, testbench, a document and a presentation for your work.